

What is AI-Assisted Coding?

Learning Objectives

By the end of this session, you should be able to:

- Explain AI-assisted coding and how it differs from traditional coding.
- Describe the mental model of vibe coding and AI code generation.
- Identify scenarios where AI-assisted coding is effective or limited.
- Recognize best practices for working with AI coding tools.
- Outline the role of tools like Visual Studio Code and GitHub Copilot.

Agenda



In this session, we'll discuss:

- AI-Assisted Coding: Concept and Workflow
- Traditional vs. AI-Assisted Development
- Mental Model: Vibe Coding and Roles
- Capabilities and Limitations of AI Coding
- Best Practices: Do's and Don'ts
- Tooling Overview: Visual Studio Code and GitHub Copilot

Understanding AI-Assisted Coding

A new way of working—not just a faster way of typing

Traditional vs. AI-Assisted Coding

Traditional Coding

- You write every line of code yourself.
- Requires deep syntax and language knowledge.
- You debug manually, line by line.
- Slow to prototype new ideas.
- The code itself is the primary output.

AI-Assisted Coding

- You describe what you want. AI writes the code.
- Focus shifts from syntax to intent and direction.
- AI helps identify and explain errors.
- Rapid prototyping from plain English descriptions.
- The outcome is the primary output.

Key Shift: You become the Director. AI becomes the Implementer.

The Mental Model of Vibe Coding

How to think about AI so you stay in control

AI Is Not a Software Engineer—It's a Probabilistic Generator

Key Definition: AI coding assistants predict the most plausible next token. They do not reason about correctness.

What AI Actually Is

- An advanced autocomplete system
- A pattern matcher trained on billions of lines of code
- A probabilistic predictor: it picks what looks most plausible
- Optimized for coherence, not correctness

What AI Is Not

- A software engineer who understands your problem
- A reasoning system that checks for logical errors
- A reliable source of truth—plausible $\neq$ correct
- A replacement for your judgment and review

The Right Mental Model: The Confident but Forgetful Junior Dev

Think of AI as a junior developer who writes fast, has seen a lot of examples, but sometimes gets details wrong and forgets what you told them 10 minutes ago.

Writes Fast

- Generates code quickly.
- Great for boilerplate, standard patterns, and scaffolding.
- Speed is its strongest suit.

Has Seen a Lot

- Trained on millions of code examples.
- Knows common patterns extremely well, but only what it has seen before.

Forgets and Drifts

- Has a limited context window.
- Forgets earlier instructions.
- Makes assumptions without telling you.
- Needs constant guidance.

This mindset naturally leads you to: verify AI output, constrain its scope, and guide it more effectively.

The Mental Model: Three Roles You Play

Navigator

- Set the direction and goal.
- Break work into clear, small steps.
- Keep the big picture in focus.
- Know what "done" looks like.

Reviewer

- Inspect every AI-generated output.
- Ask: Does this do what I asked?
- Spot gaps, errors, or drift from intent.
- Stay curious, never passive.

Decision-Maker

- Accept, reject, or redirect AI output.
- Decide when to iterate vs. move on.
- Own the final quality of the work.
- You are still accountable, always.

All three roles work together. Neglect any one and the AI will drift, fail, or mislead you.

When Vibe Coding Works and When It Doesn't

Setting realistic expectations from day one

Where AI-Assisted Coding Shines

Prototyping and Exploration

Quickly test an idea or build a proof-of-concept without deep coding knowledge. Great for early experimentation.

Repetitive Boilerplate

Generating standard patterns: data loading, API calls, file handling, CSV parsing, formatting. AI handles these fluently.

Translating Logic to Code

You know what the logic should be. AI handles the syntax and structure. Turn plain English into working Python.

Explaining Unfamiliar Code

Ask AI to walk you through code you didn't write or don't fully understand. Like having a patient tutor on demand.

Where AI-Assisted Coding Falls Short

Highly Specialized Domains

Niche libraries, proprietary systems, or cutting-edge research methods the AI hasn't seen. Outputs will be confidently wrong.

Long or Complex Codebases

AI loses context across large projects. Risks inconsistency, duplication, and forgetting earlier decisions and constraints.

Security-Critical Code

AI can produce code that looks correct but has subtle vulnerabilities. Never trust AI-generated auth, encryption, or access control without deep review.

Novel Problem Solving

When there's no clear precedent, AI generates plausible-sounding but wrong answers. Originality is a human skill.

Rule of Thumb: The more context the AI lacks, the more oversight you need.

The Do's and Don'ts

Principles that separate effective vibe coders from frustrated ones

Do's and Don'ts of AI-Assisted Coding

DO

- Verify every output the AI gives you.
- Work in small, focused steps—one task at a time.
- Ask the AI to explain code you don't understand.
- Treat AI as a collaborator, not an oracle.
- Test with inputs you can verify manually.
- Read generated code before running it in production.

DON'T

- Copy-paste AI code without reading it first.
- Ask for everything in one giant, monolithic prompt.
- Assume it is correct because it looks right.
- Use it for tasks where you cannot verify the output.
- Trust AI on security-critical or financial code without deep review.
- Confuse speed with quality; they are not the same.

Tooling Overview: VS Code + GitHub Copilot

VS Code

- Industry-standard code editor
- Free and open-source
- AI extensions plug in natively
- Works across Python and all major languages
- Integrated terminal for running code instantly

GitHub Copilot/Codex

- AI coding assistant built on OpenAI Codex
- Inline autocomplete as you type
- Chat interface for longer prompts and explanations
- Understands context across your open files
- Can write, explain, fix, and refactor code

Your Role in the Stack

- You write prompts, goals, and requirements.
- You review and accept or reject suggestions.
- You direct the iteration loop.
- You own the final quality check.
- You are the most important part of this stack.

Key Takeaways

01

You're the Director, Not the Typist

AI handles syntax and structure. Your job is to set clear goals, review output, and stay in control of the outcome.

02

Know the Sweet Spot

AI coding excels at prototyping, explaining, and repetitive tasks but needs human judgment for complex or sensitive work.

03

The Mental Model Matters

Think of yourself as navigator + reviewer + decision-maker. This mindset makes every other skill in this course click into place.

Next up: Video 2—Building the Skill: How to prompt iteratively and build real applications with AI

Summary

Here's a brief recap:

- AI-assisted coding is an approach where code is generated based on intent rather than being written line by line.
- Traditional and AI-assisted coding represent different development styles in terms of control, speed, and execution.
- The mental model of vibe coding defines how developers guide, review, and make decisions with AI.
- Capabilities and limitations describe where AI coding performs well and where it requires caution.
- Best practices establish guidelines for effectively and safely working with AI-generated code.
- Tools like Visual Studio Code and GitHub Copilot provide the environment for AI-assisted development.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

How to think iteratively with AI

Learning Objectives

By the end of this session, you should be able to:

- Explain why prompting is a structured skill and the risks of monolithic prompts.
- Describe the 4-stage staged prompting workflow (plan → requirements → tasks → implementation).
- Outline the iterative development loop (build, test, inspect, refine).
- Identify the four prompt templates used for planning, implementation, and debugging.
- Recognize how structured prompting improves control, traceability, and reliability of AI-generated code.

Agenda



In this session, we'll discuss:

- Prompting as a Structured Skill
- 4-Stage Staged Prompting Workflow
- Iterative Development Loop
- Prompt Templates Library
- Live Walkthrough: Tax Calculator

Practical Prompt Engineering for Code

Prompting is not about clever phrasing, it
is about structure and control

Common Mistake: The Monolithic Prompt

Most beginners ask for everything at once. This is the fastest way to get bloated, unreliable, hard-to-debug code.

What Beginners Do

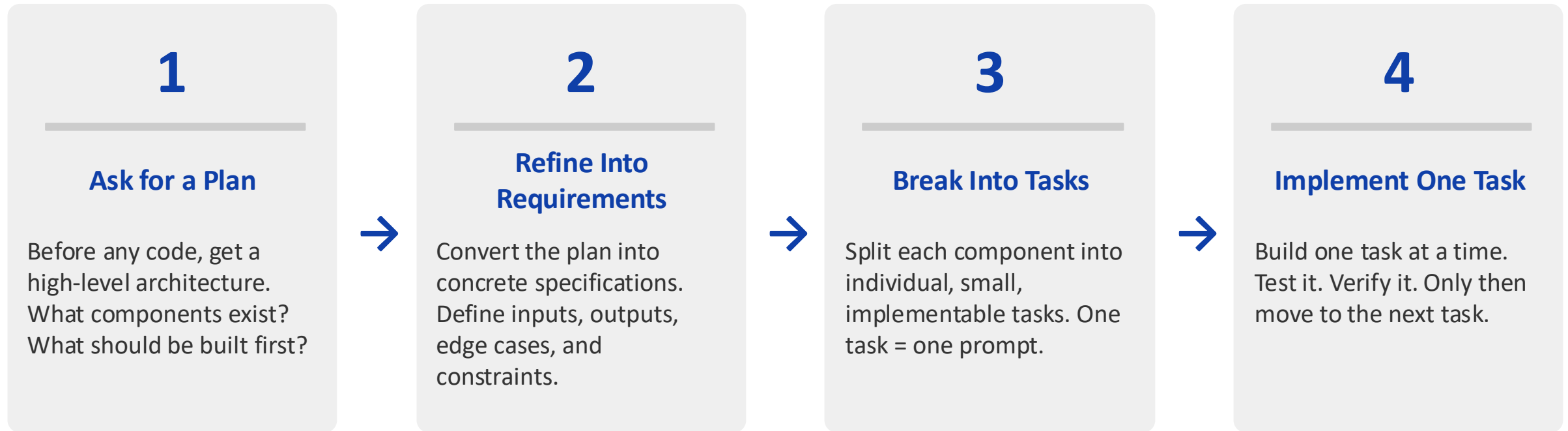
- "Build me a full tax calculator in Python with multiple tax brackets, a GUI, CSV export, previous calculation history, and an email summary feature."

What Goes Wrong

- AI tries to do everything at once—the result is bloated
- Logic is tangled, hard to trace, hard to debug
- Incomplete sections filled with stubs or placeholder code
- One prompt, one review, zero control

Large, monolithic prompts produce large, unreliable code. Smaller, focused prompts consistently yield better results.

4-Stage Staged Prompting Workflow



This staged approach gives you a checkpoint to review and verify at every step—before errors compound.

Each stage is also a decision point. If the plan is wrong, fix it before spending time on implementation.

Staged Prompting in Action: Planning the Tax Calculator

Stage 1 Prompt: "I want to build a Python tax calculator. Before writing any code, give me a high-level plan—what components do I need and what should I build first?"

AI Returns a Plan

- Component 1: Tax bracket data structure
- Component 2: `calculate_tax(income)` function
- Component 3: Input validation
- Component 4: Output formatting and display
- Component 5: (Optional) CSV export

Suggested order: 1 → 3 → 2 → 4 → 5

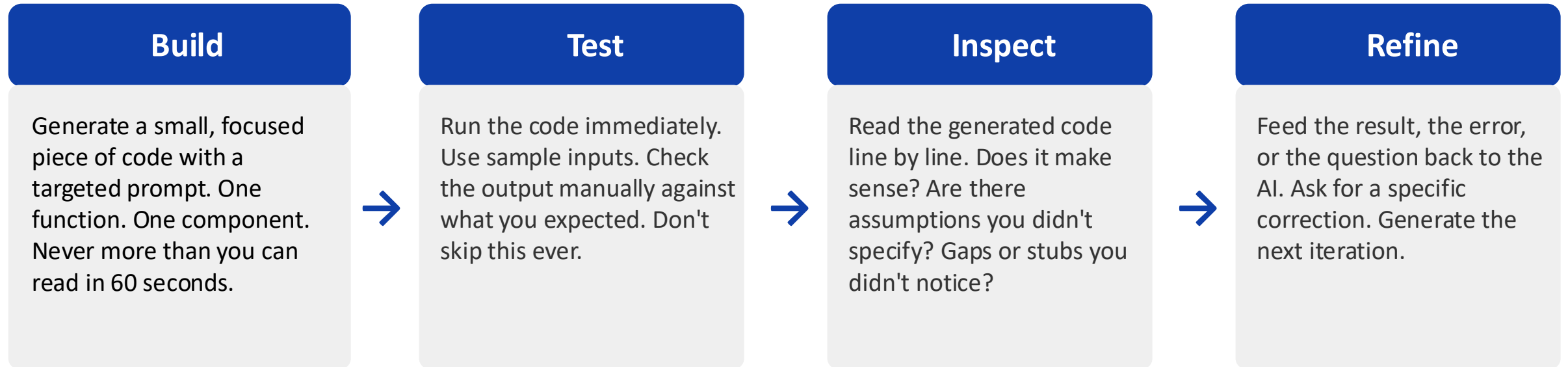
Why This Step Matters

- You can correct the plan before a single line of code is written.
- You can add missing components (e.g., "also include NI contributions").
- You set the context for all subsequent prompts in this session.
- The AI now has a shared map of the project—fewer surprises later.

The Iterative Development Loop

Build → Test → Inspect → Refine and repeat

Four-Step Iteration Loop



After Refine, go back to Build. The loop repeats until the component passes all your tests and makes sense when you read it.

Why Short Iterations Beat Long Ones—Every Time

Long Iteration (One Big Pass)

- Generate 200 lines of code in one go
- One bug hidden deep inside five functions
- Hours to find the root cause
- Hard to tell which part of the prompt caused the error
- Hard to re-prompt without rewriting everything

Short Iterations (Vibe Coding Style)

- Generate 15-20 lines for one function
- Bug appears immediately in the code you just read
- Minutes to identify and fix
- Easy to trace: this prompt → this error
- One targeted re-prompt fixes the specific issue

Short iterations keep errors small and contained. This rhythm is what allows you to use AI speed without sacrificing control.

The Prompt Templates Library

Four reusable patterns that cover 90% of what you'll need

Prompt Templates 1 and 2: Planning and Task Breakdown

Template 1: Planning Prompt

"I want to build [PROJECT]. Before writing any code, give me a high-level plan — what components do I need and what should I build first?"

- Use at the very start of every new project.
- Gets AI thinking in components, not in code.
- Gives you a shared architecture to reference throughout.

Template 2: Task Breakdown Prompt

"Here is component [NAME] from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt."

- Run once per component, after the plan is confirmed.
- Each task becomes a prompt in the implementation phase.
- Small tasks = small errors = fast debugging

Use Templates 1 and 2 before writing a single line of code. They are your project architecture phase.

Prompt Templates 3 and 4: Implementation and Debugging

Template 3: Implementation Prompt

"Implement Task [N]: [DESCRIPTION]. Use Python. Keep the code simple and add comments explaining each section. Do not implement any other tasks yet."

- One task per prompt, never bundle tasks.
- "Do not implement any other tasks yet" prevents scope creep.
- Comments help you read and verify the output.

Template 4: Debugging Prompt

"Here is the code and the error it produces: [PASTE CODE] / [PASTE ERROR]. Explain what is wrong and provide a corrected version with comments."

- Always paste both the code AND the full error message.
- "Explain what is wrong" builds your understanding. Don't skip it.
- Then re-run and verify before moving on.

Save these 4 templates. They cover: plan → requirements → implementation → debugging—the full development cycle.

Live Demo: Implementing Task 1 of the Tax Calculator

Template 3 in Action: "Implement Task 1: Create a Python function `calculate_tax(income)` using 2024/25 UK tax brackets. Add comments. Do not implement anything else yet."

What Good AI Output Looks Like

- Function is named exactly as specified: `calculate_tax(income)`
- Brackets are correct and clearly commented.
- Returns a value, not just prints it.
- No other functions added. Scope was respected.
- Comments explain each section clearly.

Your Review Checklist

- Does it run without errors?
- Does it return the right value for £20,000?
- What about £0? What about -£100?
- Any assumptions not in your prompt?

When AI Starts to Drift, How to Bring It Back

Signs That AI Is Drifting

- It builds beyond the task you asked for (scope creep).
- It contradicts decisions made earlier in the conversation.
- It forgets constraints or specifications from earlier prompts.

Technique 1: Explicit Boundaries

End every implementation prompt with: "Do not implement anything beyond this task. Do not modify any existing code."

Technique 2: Reset and Re-anchor

Start a new chat session. Paste your PROJECT_SPEC.md or a summary of decisions made so far. Re-establish shared context.

Technique 3: Summarise and Confirm

Ask: "Before we continue, summarise what we have built so far and what our next task is." This forces the AI to re-read the context.

Drift is not a failure—it is a signal. Short iterations catch it early. Long sessions let it compound.

Key Takeaways

01

Structure Beats Cleverness

Prompting is not about phrasing tricks. It is about staged workflows: plan → requirements → tasks → implement one at a time.

02

Short Loops Win

The iterative loop: Build, Test, Inspect, Refine keeps errors small and keeps you in control of the outcome.

03

Templates Save Time

Four proven templates cover planning, breakdown, implementation, and debugging. Use them on every project, every time.

Next up: Video 3—Testing and Evaluating: How to debug with AI and maintain human oversight

Summary



Here's a brief recap:

- Prompting as a structured skill defines how controlled inputs improve reliability over monolithic prompts.
- The staged workflow organizes development into sequential steps from planning to implementation.
- The iterative loop establishes a repeatable cycle for building and validating code.
- Prompt templates standardize how tasks are planned, executed, and debugged.
- The tax calculator example represents a structured application of these concepts in practice.

Learning Outcomes



You should now be able to:

- Decompose a problem into plan, requirements, tasks, and implementation stages.
- Execute the Build → Test → Inspect → Refine loop to iteratively develop code.
- Construct structured prompts using planning, implementation, and debugging templates.
- Diagnose and correct issues such as scope creep, errors, and context drift in AI outputs.
- Develop a simple application workflow (e.g., tax calculator) using staged prompting.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

How to debug with AI and maintain human oversight

Learning Objectives



By the end of this session, you should be able to:

- Explain the AI-assisted testing workflow and its role in development.
- Describe how unit tests are generated and used alongside AI-generated code.
- Outline the debugging loop for identifying and fixing failures.
- Identify key principles of code review and human oversight.
- Recognize common prompting failure modes and their impact on code quality.

Agenda

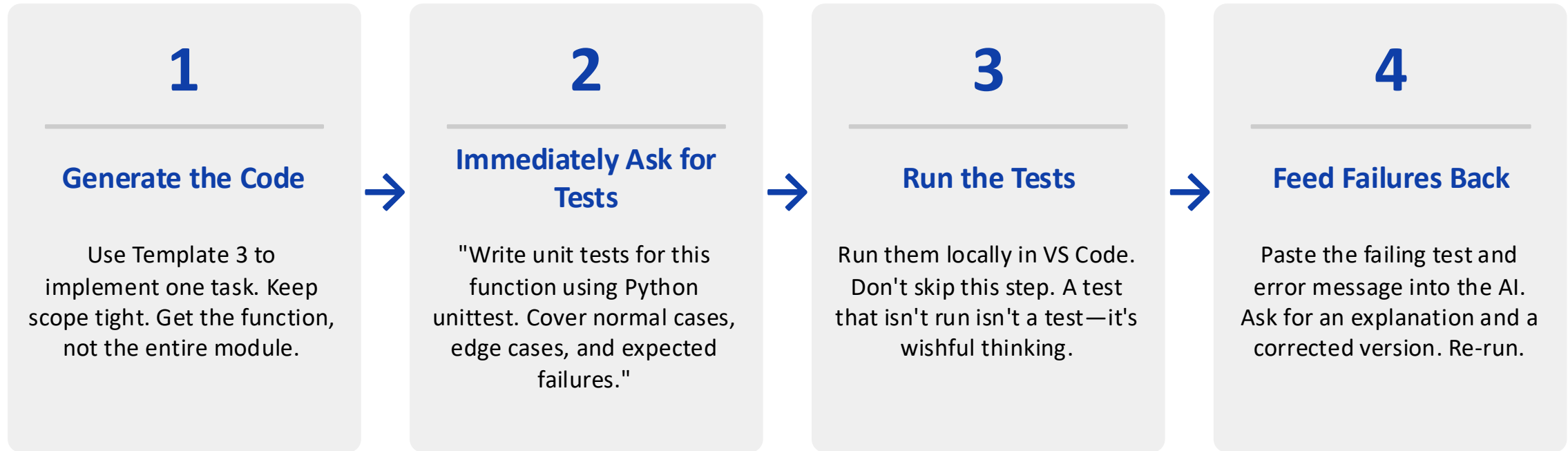
In this session, we'll discuss:

- AI-Assisted Testing Workflow
- Immediate AI-Based Test Generation After Coding
- Code Review and Human Oversight Discipline
- Failure Case Walkthroughs: Bad vs. Good Prompts
- “If AI Wrote It, You Own It” Principle

Testing and Debugging With AI

AI Excels at generating tests—but
it cannot Guarantee Correctness

AI-Assisted Testing Workflow: 4 Steps



This workflow turns the testing cycle into a collaborative loop: AI generates → you run → failures get fed back → AI corrects.

Important caveat: AI cannot guarantee correctness. It might generate tests that pass but still miss real bugs. Your job is to think about what it might have missed.

What to Ask For - What Good Tests Cover

Prompt: "Write unit tests for `calculate_tax(income)` using Python `unittest`. Cover: normal incomes, boundary incomes (at bracket edges), zero income, and negative income."

Normal Cases

- Income of £15,000: Standard earner
- Income of £30,000: Basic rate band
- Income of £60,000: Higher rate
- Verify exact tax amounts against manual calculation

Edge Cases

- Exactly at the personal allowance: £12,570
- Exactly at the basic/higher rate boundary: £50,270
- Income of £0: Should return tax of £0
- Verify bracket boundaries don't cause off-by-one errors

Failure Cases

- Negative income: Should raise `ValueError`
- String input "fifty thousand": Should raise `TypeError`
- None input: Should raise an appropriate error
- Verify error messages are clear and informative

Debugging Loop: Turn Failures into Fixes

When a Test Fails: What to Do

- Step 1: Read the error message carefully—before anything else.
- Step 2: Try to understand what it is saying.
- Step 3: Use template 4—paste both code AND full error.
- Step 4: Ask AI to explain the root cause, not just give a fix.
- Step 5: Apply the fix and re-run all tests—not just the failing one.
- Step 6: If still failing, paste the new error and repeat.

What "Explain the Root Cause" Gets You

- You learn why the code failed—not just that it failed.
- You build genuine understanding of the code over time.
- You can catch the same class of error in future code.
- You stay in control rather than blindly applying AI patches.
- You own the code, which means you need to understand it.

Never apply an AI fix without understanding what it changed. An unexplained fix to one bug often creates two new ones.

Code Review and Human Oversight

"If the AI wrote it, you own it"—the most important principle in this course

5-Point Code Review Checklist for AI-Generated Code

01

Does It Do What I Asked?

Run it with 3 different inputs you can verify manually. Check the output against your own mental calculation. Don't trust—test.

02

Are There Hidden Assumptions?

AI often makes decisions without telling you. Look for hard-coded values, assumed data types, or logic paths you didn't specify.

03

Is Anything Hard-Coded That Should Be a Variable?

Tax rates, thresholds, file paths, API endpoints—any of these hard-coded in your logic is a future bug waiting to surface.

04

Does It Handle Bad Inputs?

What happens with negative numbers? Empty strings? None values? If the AI didn't add validation, you need to ask for it explicitly.

05

Is Any Part of It Security-Sensitive?

File access, user input, network calls, authentication—anything touching these must be manually reviewed; never trust it to AI output alone.

Run this checklist on every function the AI generates. It takes 2 minutes per function and saves hours of debugging later.

"If the AI Wrote It, You Own It"—What This Actually Means

When AI-generated code goes into a project—a tax calculator, an expenses tracker, anything—you are responsible for its correctness, security, and behavior. Not the AI.

What Ownership Means

- Reading every line before it goes into your project.
- Being able to explain what each function does.
- Testing every edge case yourself.
- Fixing bugs—with or without AI help.

What Ownership Does NOT Mean

- Writing every line yourself from scratch.
- Understanding every algorithm at a deep level before use.
- Never use AI to help you fix things.
- Avoiding AI tools out of fear or guilt.

How to Build Ownership Habits

- Never copy-paste without reading first.
- Ask AI to explain code in plain language.
- Add your own comments after AI generates code.
- Run your own tests independently of AI-generated tests.

The goal is not just to produce code quickly—it is to maintain long-term quality and control over what you build.

Failure Case Walkthroughs

What goes wrong—and why—is as important as knowing what works

Failure Case 1: The Vague Prompt

The Vague Prompt

"Build a tax calculator."

What AI Returns

- Generic flat-rate tax function: No brackets.
- No error handling: Crashes on bad input.
- No structure: Just a loose function in a file.
- No comments: Impossible to verify.

The Better Prompt

"Build a Python function `calculate_tax(income)` that computes UK income tax for 2024/25. Use the following brackets: [list]. Return a dict with gross income, tax owed, and net income. Raise `ValueError` if income is negative."

What the Better Prompt Gets You

- Correct bracket structure matching real UK tax law.
- Input validation with meaningful error messages.
- Structured output you can test and verify.
- Code that matches your specification exactly.

Failure Case 2: Code Runs but Is Wrong

The most dangerous failure mode: code that produces no errors but gives subtly incorrect results, and you didn't check.

The Scenario

- AI generates `calculate_tax()`: It runs without errors.
- For income of £30,000 it returns tax of £3,486.
- The correct answer is £3,486—so it passes your eye test.
- But for income of £50,270 (bracket boundary) it gives £7,540—wrong by £200.
- Off-by-one bracket logic—invisible until you test the boundary.

How to Catch Silent Bugs

- Always test at bracket boundaries—not just round numbers.
- Verify at least 3 values manually against an authoritative source.
- Ask AI to explain how it handles boundary conditions.
- Ask AI to add explicit comments at every boundary check.
- Compare against a known correct reference (HMRC calculator).

A test that passes on round numbers is not the same as a test that is correct. Silent bugs are the most expensive kind.

Failure Case 3: Accepting AI Output Without Reading It

The pattern: Generate → copy → paste → run → "it works" → move on. Without reading a single line.

What Gets Missed

- Assumptions the AI made silently.
- Hard-coded values that will break in production.
- Security issues in input handling or file access.
- Stubs or TODOs the AI left for you to complete.
- Logic that contradicts what you actually need.

The Cost

- Bugs discovered much later—when they're expensive.
- Code you don't understand and can't maintain.
- Technical debt that compounds over time.
- False confidence in code that has never been verified.
- Loss of control over your own project.

The Fix: Read First

- Make reading the code a non-negotiable step.
- Add your own comments as you read; this proves understanding.
- Ask AI: "What assumptions did you make in this code?"
- Ask AI: "What would make this code break?"
- Never ship code you could not explain to a colleague.

Reading AI code is not optional overhead—it is the core skill that separates effective vibe coders from frustrated ones.

Prompt Quality: A Direct Comparison

Small changes in prompt structure lead to significantly different outcomes. Here is the same task—two ways.

Weak Prompt

"Fix my expenses tracker."

- No context about what is broken.
- No error message provided.
- No code provided.
- Result: AI guesses, makes changes that may break other things, and returns code you cannot verify.

Strong Prompt

"Here is my `add_expense()` function: [CODE]. When I call it with a negative amount it does not raise an error. Here is the test that fails: [TEST + ERROR]. Explain what is wrong and provide a corrected version with the validation added."

- Result: AI targets the exact problem, explains the root cause, and returns a minimal, correct fix.

Key Takeaways

01

Test Immediately, Always

Generate tests right after generating code—not later. A function without tests is an assumption, not a feature.

02

You Own What AI Writes

Read it. Understand it. Add comments. Test it yourself. The AI authored it; you are responsible for it.

03

Failure Cases Teach the Most

The vague prompt, the silent bug, the unread output—learn to recognize these patterns before they cost you hours.

Next up: Video 4—Advanced AI-Assisted Development: Context windows, stubs, deduplication, and validation hooks

Summary

Here's a brief recap:

- The AI-assisted testing workflow defines how code generation, testing, and feedback are structured.
- Test generation establishes coverage across normal, edge, and failure scenarios.
- The debugging loop provides a method to identify root causes and correct errors.
- Code review and oversight define responsibilities for validating AI-generated code.
- Failure cases highlight common prompting issues and their impact on outcomes.

Learning Outcomes



You should now be able to:

- Generate and run unit tests immediately after writing code.
- Execute the debugging loop to identify and resolve failures systematically.
- Apply a structured code review checklist to evaluate AI-generated code.
- Diagnose prompting issues such as vague instructions, silent bugs, and unverified outputs.
- Maintain ownership and accountability for all AI-generated code in a project.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Deeper Control When Working with AI

Learning Objectives

By the end of this session, you should be able to:

- Explain context window limitations and their impact on AI-assisted development.
- Describe techniques for managing state using PROJECT_SPEC.md and chunking.
- Summarize the role of stubs in structuring incomplete code.
- Examine how duplication arises in AI-generated code and its risks.
- Recognize validation hooks as a mechanism for exposing assumptions and uncertainty.

Agenda

In this session, we'll discuss:

- Context Windows and State Management
- Incomplete Code Management with Stubs
- Duplication Detection and Removal
- Validation Hooks and Traceability

Context Windows and State Management

The AI can only see a finite window—
and earlier details degrade

Context Window Constraint

The AI has no persistent memory between sessions. Within a session, as the conversation grows, earlier details degrade or disappear entirely.

What Happens Without Management

- AI forgets requirements stated 20 prompts ago
- It contradicts decisions already agreed and built
- It generates duplicate logic because it can't see the full codebase
- Inconsistencies accumulate session after session
- Your codebase diverges from your original plan silently

Three Techniques to Manage Context

- Technique 1: Work in small, well-defined chunks per session
- Technique 2: Externalise state into PROJECT_SPEC.md
- Technique 3: Reset context and re-anchor at the start of new sessions

Manage context deliberately, not accidentally. The developers who do this have fewer surprises, fewer bugs, and faster progress.

Your Most Powerful Tool: The PROJECT_SPEC.md File

What to Put in Your PROJECT_SPEC.md

What you are building and why · Key components and their current status · Decisions already made · Constraints and rules the AI must follow · Current task and next task

How to Use It

Paste the relevant section at the start of every new session. "Here is our project context. We are currently working on Task 3. Do not modify Tasks 1 or 2."

When to Update It

After every session. Mark completed tasks. Record any new decisions. Add any constraints that emerged. Treat it as your source of truth.

What It Replaces

The mental model of "the AI remembers." It doesn't. This file is your AI's external memory—and yours too across long projects.

Your PROJECT_SPEC.md turns the AI from a forgetful collaborator into one that always starts with the full picture.

Managing Incomplete Code (Stubs)

Stubs are not a problem—they are a structured to-do list

Reframe Stubs: Frustration to Roadmap

When AI reaches uncertainty or practical limits, it fills gaps with stubs—placeholder functions marked with TODO. This is normal. The mistake is being surprised by it.

The Problem Approach

- Ask AI to generate everything in one go
- Receive code with hidden stubs and incomplete logic
- Run it—it fails in unexpected places
- Spend time hunting for what is incomplete

The Reframe

- Ask AI to scaffold with intentional stubs first
- Each stub = one task = one implementation prompt
- Clear TODO comments = visible, trackable progress
- Stubs become your sprint board

The Prompt to Use

"Before implementing, create a skeleton of the full application with all functions defined but not yet implemented. Use clear TODO comments explaining what each function should do and what inputs and outputs are expected."

A skeleton with clear TODOs is more valuable than rushed, half-working code. It is a map of your project.

Detecting and Managing Duplication

AI optimises locally—you must manage the global picture

Duplication...and How to Fix It

AI generates code based on each prompt in isolation. It has no global view of your codebase. Over time this creates duplicate logic—the same function written twice, in different places, slightly differently.

Why Duplication Is Dangerous

- A bug fix in one copy doesn't fix the other
- The two copies may diverge over time, producing inconsistent behaviour
- Codebase becomes confusing—which version is the real one?
- Long-term instability that is hard to trace back to a source

Two Techniques to Control It

Technique 1: Periodic Refactoring Prompt (every 3-4 tasks):
"Review the code below for duplicated logic or functions. Suggest how to refactor it to eliminate duplication while keeping behaviour identical."

Technique 2: Inline Flags:

Add comments like: "NOTE: Similar logic exists in `validate_input()`—consider merging before next refactor."

Never go more than 4 implementation tasks without running a duplication check. Duplicate logic compounds faster than you expect.

Validation Hooks and Traceability

Make the AI tell you what it might have gotten wrong

Embedding Validation Hooks in AI-Generated Code

A validation hook is a comment embedded in the code that makes assumptions, uncertainties, and incomplete sections visible—rather than hidden.

The Prompt That Creates Hooks

"After generating the code, add inline comments that identify: (1) any assumptions you made that I should verify, (2) any parts you are uncertain about, (3) any sections that are incomplete or require follow-up."

Example Output

```
# ASSUMPTION: Using 2024/25 UK brackets—verify these are correct  
# ASSUMPTION: No consideration for Scottish rates  
# TODO: Add handling for the personal savings allowance
```

What Validation Hooks Give You

- A map of where to review — not a search for problems
- Visible assumptions that you can verify or challenge
- Incomplete sections that become explicit tasks
- A structured inspection process instead of blind trust

Resolving Hooks Over Time

Verify → remove the ASSUMPTION comment. Complete → remove the TODO.
Clean code = resolved hooks.

Putting Advanced Techniques Together

1

Start Fresh Session

Paste PROJECT_SPEC.md.
Re-anchor AI context. State
current task explicitly.



2

Scaffold with Stubs

Ask for skeleton first. All
functions defined, none yet
implemented. Review the
structure before building.



3

Implement with Hooks

Build one stub at a time. Ask
AI to annotate assumptions
and uncertainties after each
implementation.



4

Audit and Refactor

Every 3-4 tasks: run
duplication check, resolve
validation hooks, update
PROJECT_SPEC.md.

These four techniques together turn AI from a novelty into a reliable production tool for real Python projects.

They all address the same root problem: AI has no global awareness. These techniques give it one—artificially, but effectively.

Key Takeaways

01

Manage Context on Purpose

Use PROJECT_SPEC.md. Work in small session chunks. Reset and re-anchor regularly. Don't assume the AI remembers.

02

Stubs Are a Roadmap

Ask for intentional scaffolding. Each stub is a task. Each TODO is a checkpoint. Visible gaps are manageable gaps.

03

Audit and Validate Consistently

Run deduplication checks every 3-4 tasks. Use validation hooks to surface assumptions. Clean code means resolved hooks.

Next up: Video 5—Summary and Key Takeaways: The repeatable playbook and the honest costs of over-reliance

Summary

Here's a brief recap:

- Context windows define how AI retains and loses information across sessions.
- State management establishes external methods to maintain continuity and consistency.
- Stubs represent structured placeholders that guide incremental development.
- Duplication highlights the risk of locally generated code without global awareness.
- Validation hooks provide visibility into assumptions, uncertainties, and incomplete logic.

Learning Outcomes

You should now be able to:

- Manage AI context using PROJECT_SPEC.md, chunking, and session resets.
- Structure development workflows by creating and implementing stubs incrementally.
- Identify and refactor duplicated logic across AI-generated code.
- Embed and resolve validation hooks to verify assumptions and completeness.
- Maintain a consistent and traceable codebase across multiple AI-assisted sessions.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Key Take Aways

Repeatable Vibe Coding Playbook: 8 Steps

1	2	3	4	5	6	7	8
Define the Goal	Ask for a Plan	Convert to Tasks	Implement One Task	Add Validation Hooks	Generate Tests	Debug and Review	Refactor and Repeat
One clear paragraph. What does it do? Who uses it? What does it NOT do? No ambiguity.	Ask AI for architecture before writing any code. Get components and suggested build order.	Break each component into individual tasks small enough for one prompt each.	Use Template 3. One task per prompt. Never bundle. Test before moving to the next.	Ask AI to annotate assumptions and uncertainties inline as code comments.	Ask AI to write unit tests immediately after each function is generated.	Run tests. Feed failures back using Template 4. Read code. Apply review checklist.	Every 3-4 tasks: check for duplication, resolve TODO hooks, update PROJECT_SPEC.md.

Apply this loop to any Python project. The playbook works for a tax calculator, an expenses tracker, and anything you build next.

3 Risks Every Vibe Coder Must Manage

Risk 1: Shallow Understanding

If you cannot explain a function you generated, in plain language, to a colleague, you do not own it.

Fix: Ask AI to explain every piece of generated code. Read it until you can paraphrase it. Add your own comments. Slow down to go fast.

Risk 2: Accumulated Technical Debt

Stubs left unresolved. Duplicate logic never cleaned up. Validation hooks never verified. Each one is a future bug at compound interest.

Fix: Treat every TODO and ASSUMPTION comment as a debt entry. Resolve on a schedule—not "when I have time."

Risk 3: False Sense of Productivity

Code that runs is not the same as code that works correctly, securely, and sustainably.

Fix: Measure productivity by tests passing and features verified, not by lines of code generated. Output is not the same as quality.

Rule of Thumb: If you could not maintain this code without the AI, you built it too fast.

The Balance: Speed With Comprehension

The goal of this course is not to help you generate code quickly. It is to help you build things that work, that you understand, and that you can improve.

What to Keep

- Using AI to move faster on well-understood, repeatable tasks
- Using AI as a collaborator: plan together, implement together, review yourself
- Using AI to learn, ask for explanations, not just code
- The iterative loop, the templates, the playbook

What to Watch

- Moving so fast that you stop understanding what you've built.
- Treating AI as a search engine for correct code. It isn't.
- Stacking features before earlier features are verified.
- Forgetting that you, not the AI, are responsible.

AI is a multiplier. It amplifies the habits of the developer using it—good habits and bad ones equally.

What to Do Next: Three Practice Projects

Practice Project 1

Extend Your Tax Calculator

- Add support for different tax years (2023/24 vs. 2024/25).
- Add a command-line interface, accept income as user input.
- Export results to a formatted text file.
- Use the full 8-step playbook: plan first, scaffold, implement, test.

Practice Project 2

Build the Expenses Tracker from Scratch

- Add expense entries (amount, category, date).
- Show a summary by category.
- Save and load data from a CSV file.
- No hand-holding—full playbook, independent build.

Practice Project 3

Choose Your Own Project

- Pick something you actually need or want to build.
- Start with the Planning Prompt. No code before a plan.
- Apply every technique from this course.
- Notice where AI helps and where you need to take control.

The skill of AI-assisted development compounds quickly. Each project will be faster, cleaner, and more controlled than the last.

Your Goal: After three projects, you should be able to explain every function you've built. That's mastery.

Everything This Course Has Given You

Video 1: Mental Model

AI is a probabilistic generator, not a software engineer. You are the Director. Know when to use it and when to take manual control.

Video 2: Building the Skill

Staged prompting: Plan → Requirements → Tasks → Implement. The iterative loop. Four reusable prompt templates.

Video 3: Testing and Debugging

Generate tests immediately. Feed failures back. Apply the 5-point review checklist. Own everything AI writes.

Video 4: Advanced Techniques

PROJECT_SPEC.md for context. Intentional stubs as roadmaps. Deduplication audits. Validation hooks for traceability.

The 8-step playbook ties it all together. Define → Plan → Tasks → Implement → Hooks → Tests → Debug → Refactor. Repeat.

Welcome to the new way of building software. Now go build something.



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING